

08 – Advanced SQL

Back to advanced SQL

Jérôme Landré – jerome.landre@ju.se

(Several slides have been created by Jasmin Jakupovic)

OUTLINE

1. Transactions
2. ACID
3. Indexing
4. Stored procedures
5. Triggers
6. Views

1. Transactions

Definitions

A database can be accessed by several different users at the same time.

Each user may start one or several transactions towards the database.

It is important for the database system to keep track of what transaction happens, when, where and what has been done

What is it?

A transaction is a series of actions, in the form of queries, performed to achieve a specific goal

e.g. Transfer of an amount of money from one bank account to another.

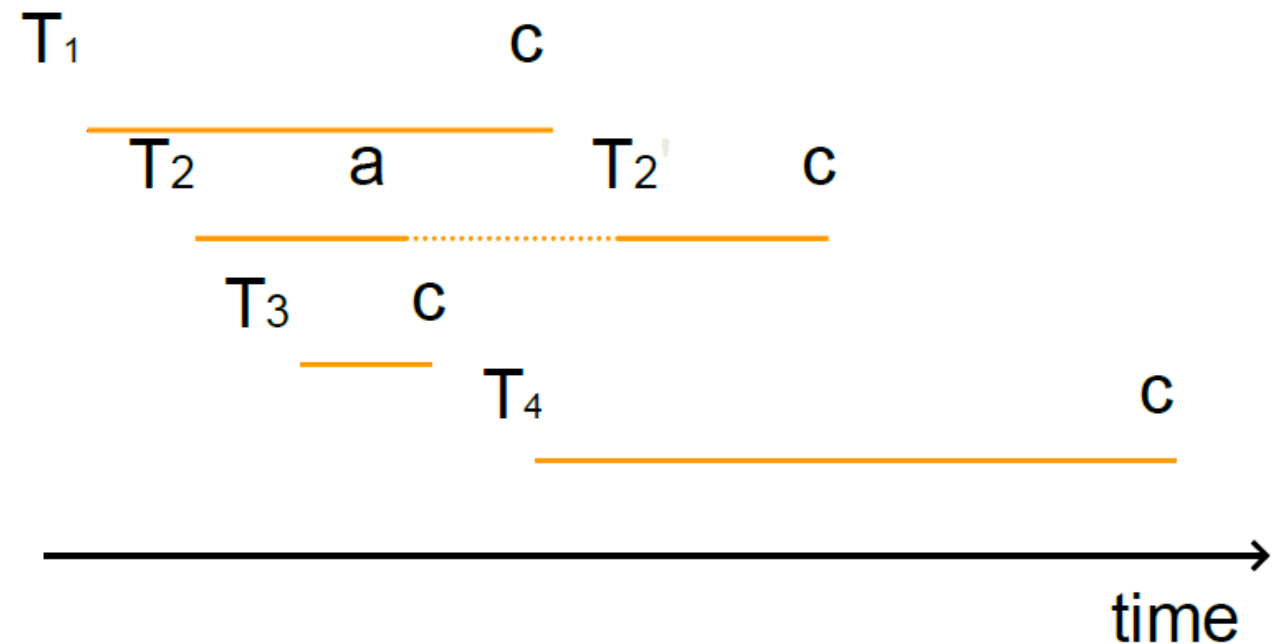
1. Transactions

How does it work?

A transaction (T) may either terminate with a commit (c) or abort (a). The transaction commits if it was successful and aborts otherwise.

How does it work?

If a transaction is aborted it will be restarted later either automatically, by DBMS, or manually by the user.



1. Transactions

In a multi-user system transactions come from different users using e.g.:

- Applications
- Stored Procedures
- Scripts

Efficiency

For the DBMS to handle this efficiently operations from different transactions, they have to be **interleaved**.

During I/O operations for a transaction the CPU may switch to another transaction.

The theory for transaction management is mainly developed for interleaving in a single CPU.

1. Transactions

Keywords

- **Begin_transaction**
- **End_transaction**
- Read_item
- Write_item
- Commit
- Rollback

Keywords

In practice a transaction can only end in either a **commit** (validate changes) or a **rollback** (return to previous state).

End of transaction

we use the "End_transaction" statement for marking where the coding of the transaction is ending.

1. Transactions

Keywords

- Begin_transaction
- End_transaction
- **Read_item**
- **Write_item**
- Commit
- Rollback

Keywords

In practice a transaction can only end in either a **commit** (validate changes) or a **rollback** (return to previous state).

Read and Write

Read_item:

1. Locate the block on disk
2. Copy the block to a page in primary memory
3. Copy the data item to a variable in the Application

Write_item:

1. Locate the block where the data should be copied
2. If the primary memory buffer does not contain the page, copy it to an available page
3. Copy data from the variable to the available page
4. Copy page to block on disk

1. Transactions

Keywords

- Begin_transaction
- End_transaction
- Read_item
- Write_item
- **Commit**
- **Rollback**

Keywords

In practice a transaction can only end in either a **commit** (validate changes) or a **rollback** (return to previous state).

Commit and Rollback

Commit:

Purpose: Saves all changes made during the current transaction permanently to the database.

Effect: Once committed, changes cannot be undone; they become visible to all other database sessions.

Rollback:

Purpose: Undoes all changes made during the current transaction, reverting the database to its state before the transaction began.

Effect: All modifications are discarded, and the database returns to its previous consistent state.

Problems with transactions

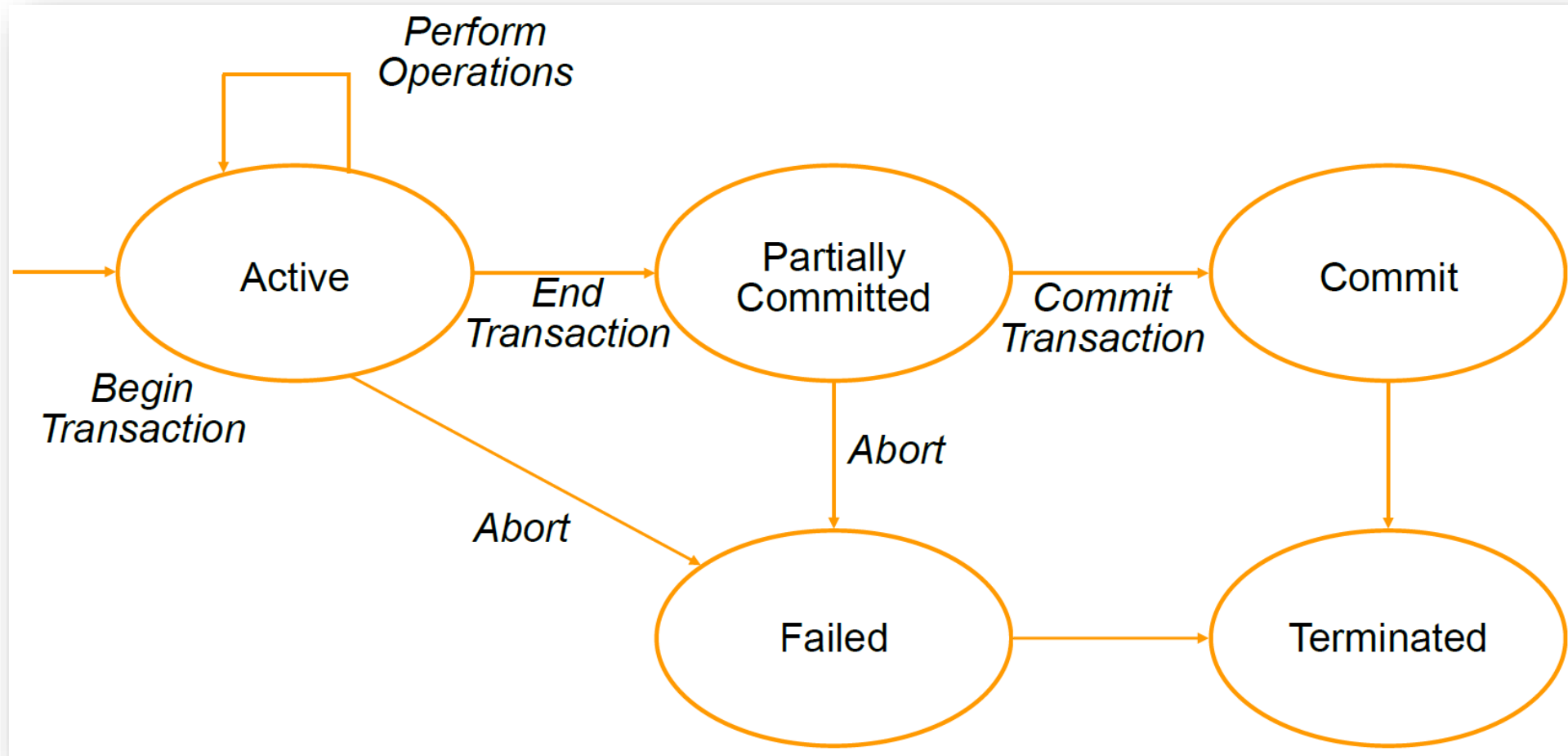
- Lost Update Problem
- Uncommitted Dependency Problem
- Inconsistent Analysis Problem

One transaction overwrites the information written by another transaction.

The value of a data item written by one transaction, that later on performs a rollback, is read by a second transaction before the DBMS has changed the value back.

The result from a transaction, that is only regarding some data items, may become erroneous because another transaction is updating the same data items.

Transaction States



2. ACID

Desirable properties for transactions:

Atomicity - the transaction has to be performed in its entirety or not at all.

Consistency - the transaction should leave the database in a consistent state, all data should be correct.

Isolation - the interleaving of operations from different transactions should be performed in such a way as if the transactions were to run one by one.

Durability - the effect of a committed transaction should not be lost by any failure.

2. ACID

Schedule

Is an order in which operations from different transactions are interleaved with each other.

We can have several schedules involving the same set of transactions, one schedule for each ordering of the operations.

Internal ordering within each transaction must not be changed.

... used for ...

Recoverability and Serialisability

Recoverability: Find the type of schedules from which it is possible to recover the database to a consistent state of a failure. (A & D)

Serialisability: Find the type of schedules which will give a correct result. (C & I)

3. Indexing

Why do we need indexes?

As a human being, I want to find the Person with lastname 'Martinez' in a list, what will I do?

If one row search is 100ns, how long will it take?

$$\begin{array}{rcccc} 100 & * & 1\,000\,000 & = & 100\,000\,000 \\ & & \text{s} & & \text{ms} & & \text{ns} \\ & & 100 & & 000 & & 000 \end{array}$$

(id, firstname, lastname, city)

(1, John, Smith, New York)

(2, Emma, Johnson, London)

(3, Michael, Williams, Chicago)

(4, Sophia, Brown, Paris)

(5, William, Jones, Sydney)

(6, Olivia, Garcia, Toronto)

...

(999997, James, Miller, Berlin)

(999998, Ava, Davis, Tokyo)

(999999, Robert, Rodriguez, Madrid)

(1000000, Isabella, Martinez, San Francisco)

3. Indexing

Is there a better way?

Yes: organizing the data into a **TREE**

(id, firstname, lastname, city)

- (1, John, Smith, New York)
- (2, Emma, Johnson, London)
- (3, Michael, Williams, Chicago)
- (4, Sophia, Brown, Paris)
- (5, William, Jones, Sydney)
- (6, Olivia, Garcia, Toronto)

- ...
- (999997, James, Miller, Berlin)
- (999998, Ava, Davis, Tokyo)
- (999999, Robert, Rodriguez, Madrid)
- (1000000, Isabella, Martinez, San Francisco)

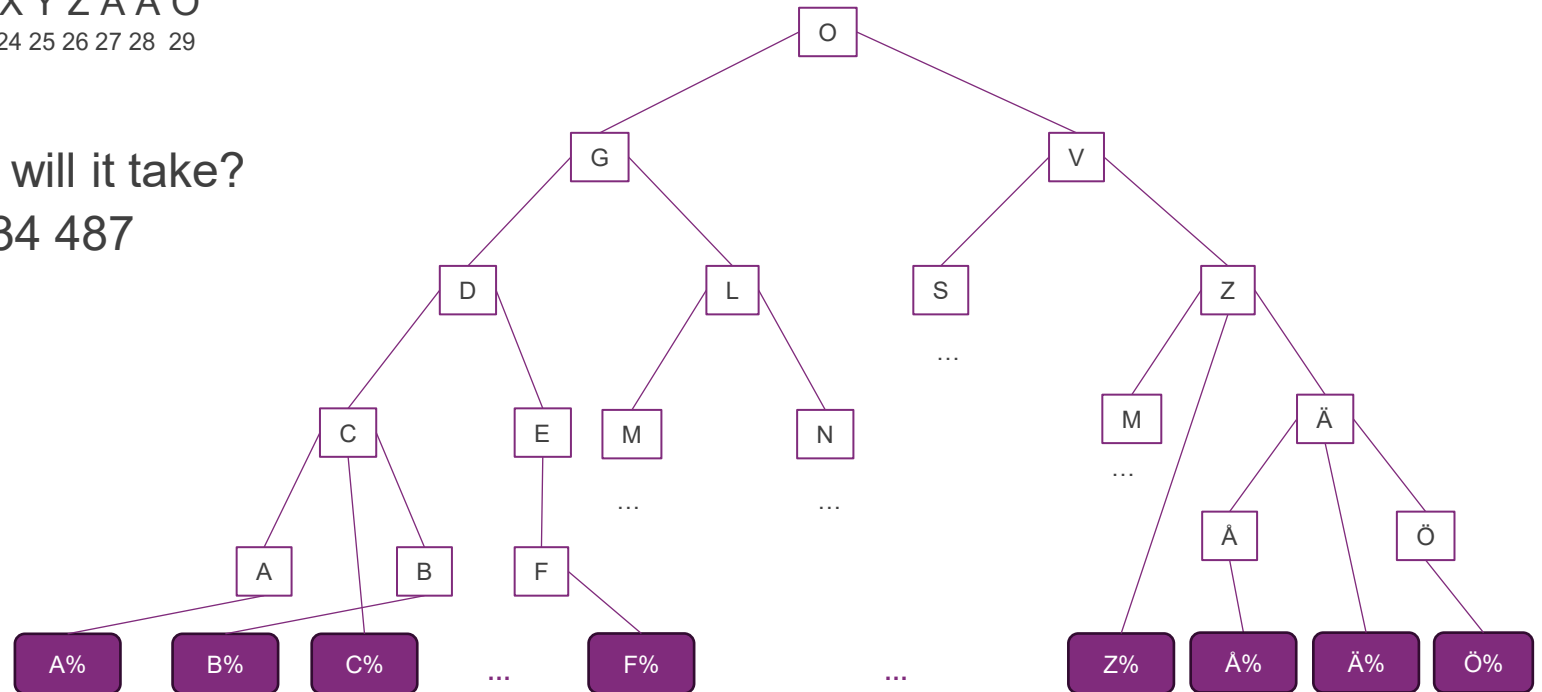
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z Å Ä Ö
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29

If one search is 100ns, how long will it take?

$$5 + 1000000/29 \sim 5 + 34\,482 = 34\,487$$

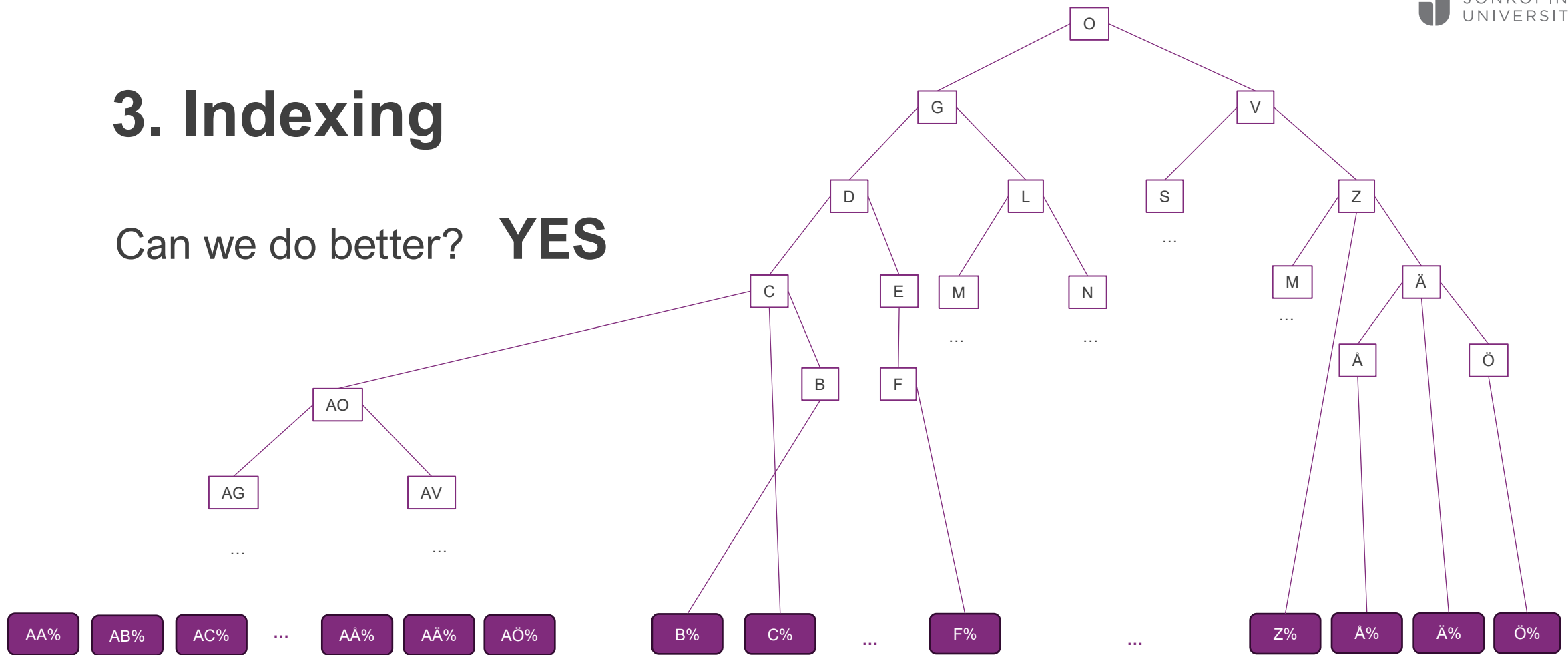
$$100 * 34\,487 = 3\,448\,700$$

s	ms	ns
3	448	700



3. Indexing

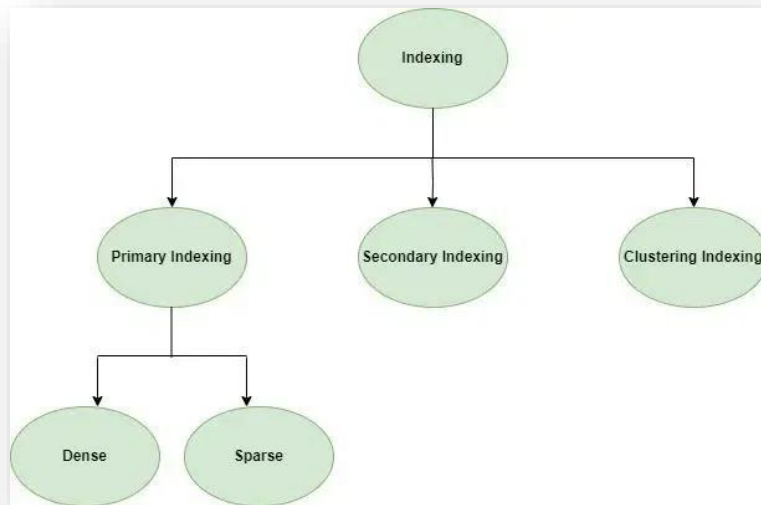
Can we do better? **YES**



3. Indexing

Speed up your SQL queries

Is built on a single field or a combination of several fields. A separate table containing only the indexing fields and pointers to blocks or records for the data pages.



Types and Categories

Types of indexes:

- Primary indexes
- Clustering indexes
- Secondary indexes

Categories of indexes:

- Dense
- Sparse (Non-dense)

Types of indexes

Primary Index - primary key or candidate key

Clustering Index - fields that allow duplicate values

Secondary Index - index on any non-ordering field

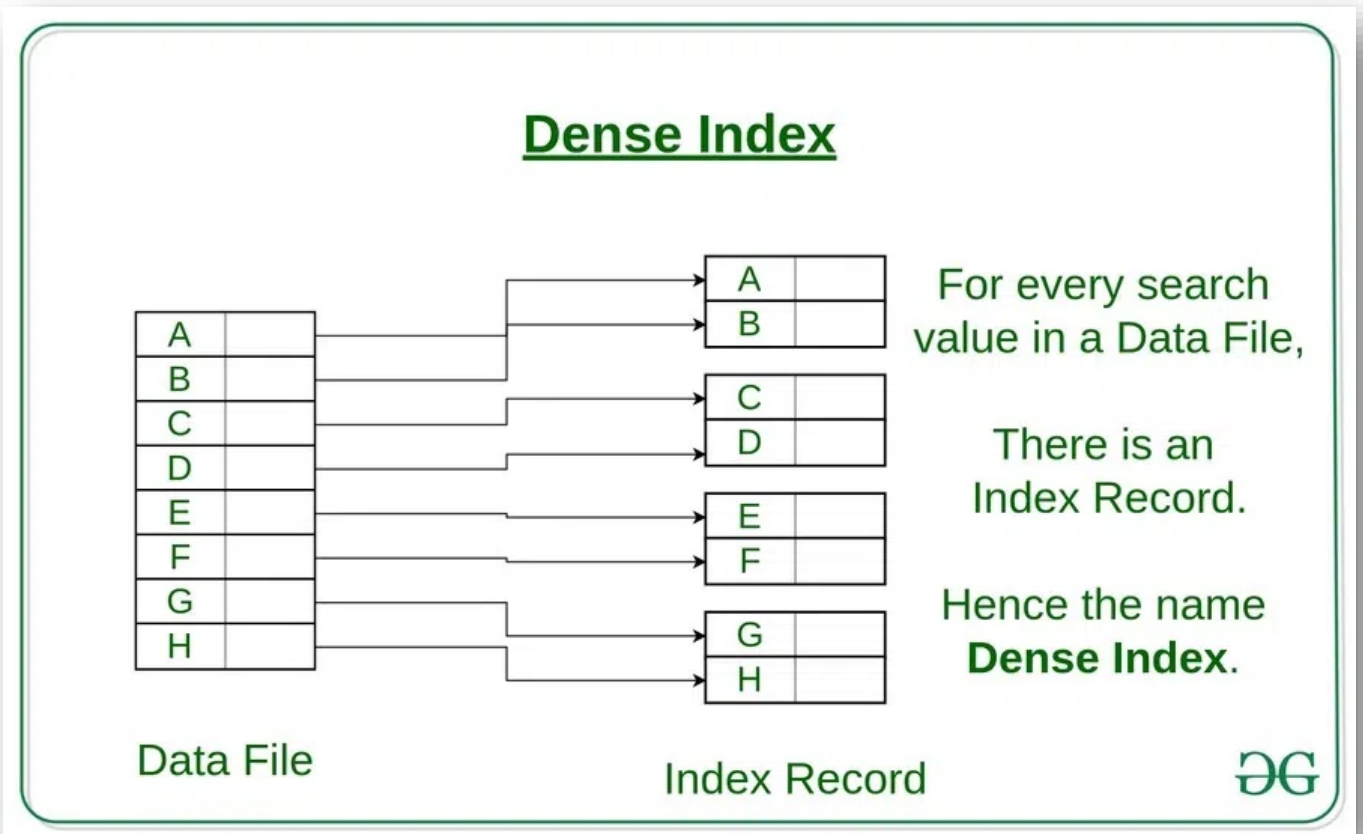


Physically sorts the order of the records in the DB

3. Indexing

Dense index

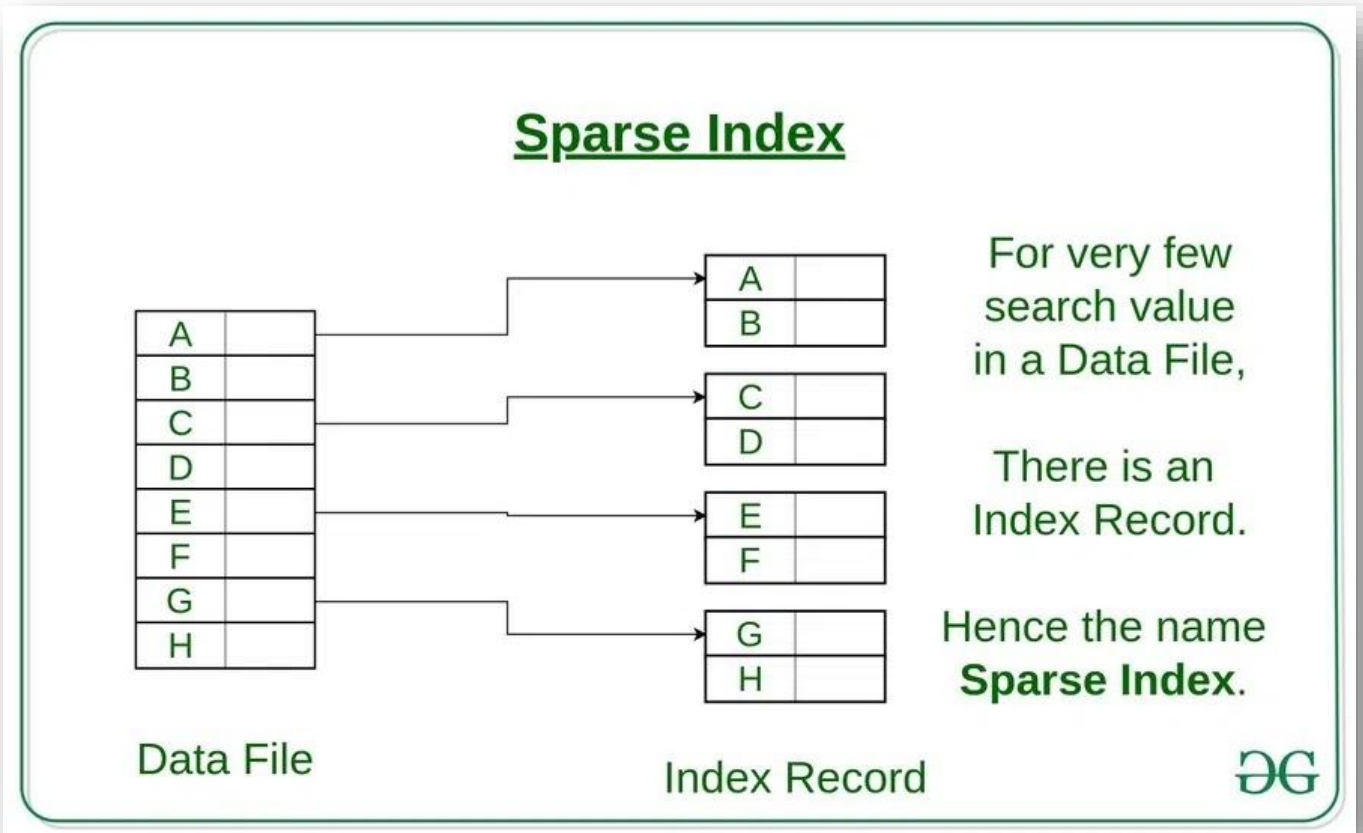
- A dense index appears for every search key value in a file.
- This record contains search key value and a pointer to the actual record.



3. Indexing

Sparse index

- Index records created for some of the records.
- To locate a record, we find the index record with the largest search key value less than or equal to the search key value we are aiming for.



3. Indexing

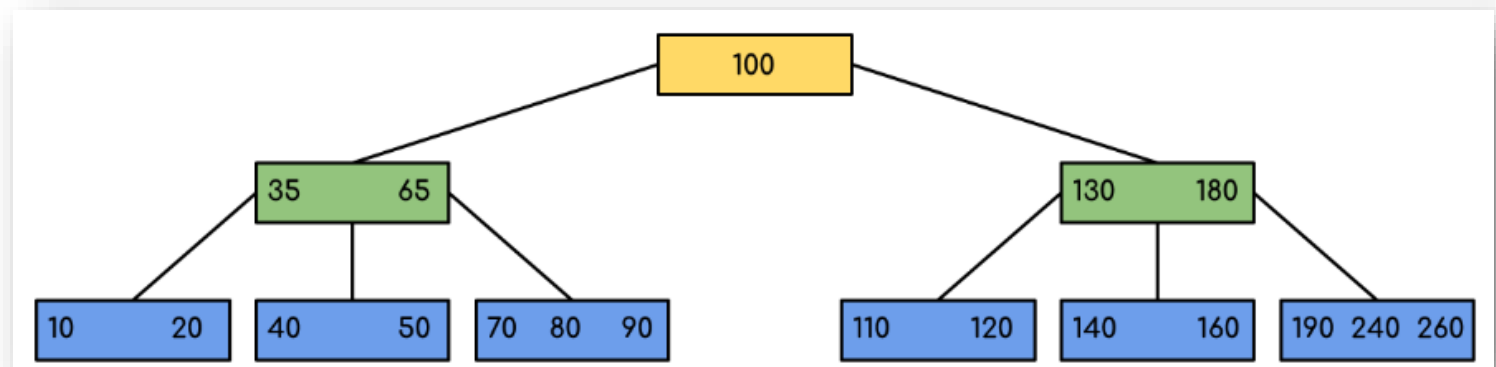
Example of indexing: Numbers

- If you want to find in a list of prices of 19 products, the products sold 120 kr:
SELECT * FROM products WHERE price=120;

Without index: you do things sequentially. You will go through all the products to test so, it will need 19 operations (comparisons).

[10, 20, 35, 40, 50, 65, 70, 80, 90, 100, 110, 120, 130, 140, 160, 180, 190, 240, 260]

With an index: you will go through a pre-sorted tree, needing only a few operations to locate the desired product (here 4 steps): >100, <130, two comparisons.



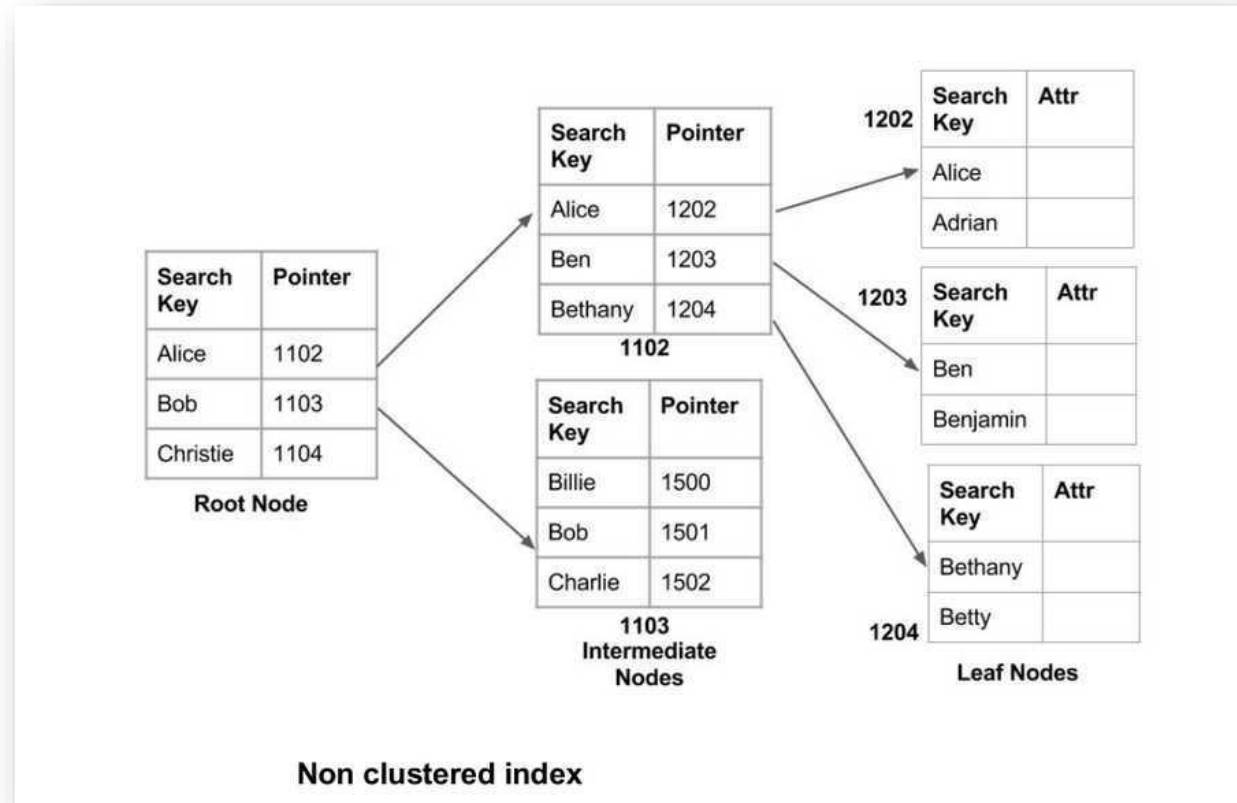
3. Indexing

Example of indexing: Strings

- If you want to find in a list of 1000 persons, the ones that have "Betty" as first name: **SELECT * FROM persons WHERE firstName="Betty"**

Without index: you do things sequentially. You will go through all the 1000 persons to test so it will need 1000 operations (comparisons).

With an index: you will go through a pre-sorted tree, needing only a few operations to locate the desired person (here 2+ steps): <"Bob", >"Bethany".



4. Stored Procedures

- A stored procedure groups one or more SQL statements that are stored in an *object*.
- They are used for security reasons to avoid SQL injections in the applications
- Stored procedures are often, once run for the first time, stored in the cache memory. This allows them to execute much faster the next time they are "*called*".

From simple to advanced

- Stored procedures can do simple things, but can also be programmed to handle exceptions which makes them more advanced
- They are most often used with control of flow statements such as IF ... ELSE, WHILE, BREAK, BEGIN, END and CONTINUE.

4. Stored Procedures

How to declare Stored Procedures?

```
SELECT uniName, uniRating
FROM university
ORDER BY uniRating
```

```
CREATE PROCEDURE uniProc
AS
BEGIN
    SELECT uniName, uniRating
    FROM university
    ORDER BY uniRating
END;
```

Begin ... End

This statement defines the statement block or a batch and can also be nested.

```
BEGIN
    DECLARE @name VARCHAR(MAX);

    SELECT TOP 1
        @name = product_name
    FROM
        production.products
    ORDER BY
        list_price DESC;

    IF @@ROWCOUNT <> 0
    BEGIN
        PRINT 'Most expensive ' + @name
    END
    ELSE
    BEGIN
        PRINT 'No product found';
    END;
END
```

4. Stored Procedures

```
CREATE PROCEDURE uniProc
AS
BEGIN
    SELECT uniName, uniRating
    FROM university
    ORDER BY uniRating
END;
```

- uniProc is the procedure name.
- Execute it by typing EXEC + name.
- Edit it by using ALTER PROCEDURE.
- Delete with a DROP PROC + name.

Execute it!

```
EXECUTE sp_name;
/* OR */
EXEC sp_name;
/* In our case */
EXEC uniProc;
```

Modify it!

```
ALTER PROCEDURE uniProc
AS
BEGIN
    SELECT uniName, uniRating
    FROM university
    ORDER BY uniRating ASC
END;
```

Delete it!

```
DROP PROCEDURE sp_name;
```


5. Triggers

When X happens, do Y

Special stored procedures that are automatically *triggered* (executed) in response to a database object, database, and server event.

- There are three main types of triggers:
 - DML
 - DDL
 - LOGON

Types of triggers

- Data Manipulation Language (DML) Triggers
- Data Definition Language (DDL) Triggers
- Logon Triggers

DML Triggers are invoked automatically in response to INSERT, UPDATE, and DELETE events against tables.

DDL Triggers are invoked automatically in response to CREATE, ALTER, and DROP statements.

LOGON Triggers are invoked automatically in response to LOGON events on a database.

5. Triggers

How to?

```
CREATE TRIGGER triggerName
ON tableName
AFTER { [INSERT], [UPDATE], [DELETE] }
[NOT FOR REPLICATION]
AS
{SQL Statements}
```

1. triggerName - user-defined name for the new trigger
2. tableName - which table the trigger applies to
3. AFTER - what event does the trigger happen
4. NOT FOR REPLICATION - tells the SQL Server not to trigger for the replication process
5. SQL Statements - what will happen when it's triggered

Example of a trigger:

```
CREATE TRIGGER bookcopytrig
ON bookcopy
FOR INSERT, UPDATE, DELETE
AS IF UPDATE(isbn)
BEGIN
    DECLARE @insertedisbn CHAR(20);
    DECLARE @deletedisbn CHAR(20);
    IF EXISTS(SELECT isbn FROM INSERTED)
    BEGIN
        SELECT @insertedisbn = (SELECT isbn FROM INSERTED);
        EXECUTE updatebook @insertedisbn;
    END
    IF EXISTS(SELECT isbn FROM DELETED)
    BEGIN
        SELECT @deletedisbn = (SELECT isbn FROM DELETED);
        EXECUTE updatebook @deletedisbn;
    END
END
```

Explanations:

```
CREATE TRIGGER bookcopytrig
ON bookcopy
FOR INSERT,UPDATE,DELETE
AS IF UPDATE(isbn)
BEGIN
    DECLARE @insertedisbn CHAR(20);
    DECLARE @deletedisbn CHAR(20);
    IF EXISTS(SELECT isbn FROM INSERTED)
    BEGIN
        SELECT @insertedisbn = (SELECT isbn FROM INSERTED);
        EXECUTE updatebook @insertedisbn;
    END
    IF EXISTS(SELECT isbn FROM DELETED)
    BEGIN
        SELECT @deletedisbn = (SELECT isbn FROM DELETED);
        EXECUTE updatebook @deletedisbn;
    END
END
```

The trigger is first created, and concerns a table: bookcopy
Select what triggers the action. In our case it is an INSERT, UPDATE or DELETE.

If it is correct, do an update on the ISBN

Declare two new variables, @insertedisbn and @deletedisbn which will be used to retrieve the newly inserted/deleted isbn.

Checks if anything has recently been inserted.

If something was inserted, it takes the latest ID from the temporary INSERTED table.

Finally, it executes a stored procedure as a result of the insert.

5. Triggers

Trigger with Stored Procedures:

It is a neat/clean way to automate your code. You can simply create some stored procedures and call them when something happens.

Temporary tables:

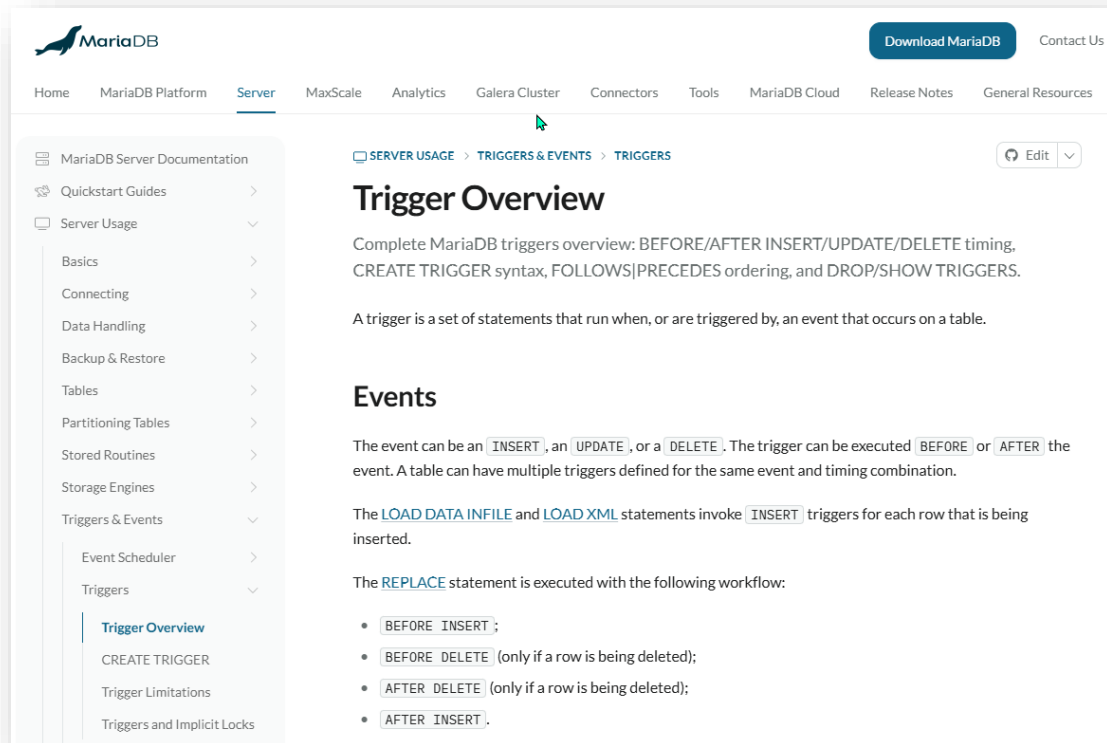
When you do an insert on a specific table you can get the latest value for a specific column. This can be very useful when we need to use that exact record later on.

Declare:

We can always declare new, temporary variables, to be used within the trigger.

5. Triggers

Triggers exist in MariaDB and MySQL:



The screenshot shows the MariaDB Server Documentation website. The navigation bar includes links for Home, MariaDB Platform, Server (selected), MaxScale, Analytics, Galera Cluster, Connectors, Tools, MariaDB Cloud, Release Notes, and General Resources. A sidebar on the left lists various documentation topics, with 'Triggers' expanded under 'Triggers & Events'. The main content area is titled 'Trigger Overview' and provides a comprehensive overview of triggers, including their syntax, timing, and usage. It explains that a trigger is a set of statements that run when an event occurs on a table. The page also covers events, such as INSERT, UPDATE, and DELETE, and how triggers can be executed before or after the event. A list of trigger types is provided, including BEFORE INSERT, BEFORE DELETE, AFTER DELETE, and AFTER INSERT.

Trigger Overview

Complete MariaDB triggers overview: BEFORE/AFTER INSERT/UPDATE/DELETE timing, CREATE TRIGGER syntax, FOLLOWS|PRECEDES ordering, and DROP/SHOW TRIGGERS.

A trigger is a set of statements that run when, or are triggered by, an event that occurs on a table.

Events

The event can be an `INSERT`, an `UPDATE`, or a `DELETE`. The trigger can be executed `BEFORE` or `AFTER` the event. A table can have multiple triggers defined for the same event and timing combination.

The `LOAD DATA INFILE` and `LOAD XML` statements invoke `INSERT` triggers for each row that is being inserted.

The `REPLACE` statement is executed with the following workflow:

- `BEFORE INSERT`;
- `BEFORE DELETE` (only if a row is being deleted);
- `AFTER DELETE` (only if a row is being deleted);
- `AFTER INSERT`.

Creating a Trigger

Here's a simple example to demonstrate a trigger in action. Using these two tables as an example:

```
CREATE TABLE animals (id MEDIUMINT(9)
NOT NULL AUTO_INCREMENT,
name CHAR(30) NOT NULL,
PRIMARY KEY (`id`));

CREATE TABLE animal_count (animals INT);

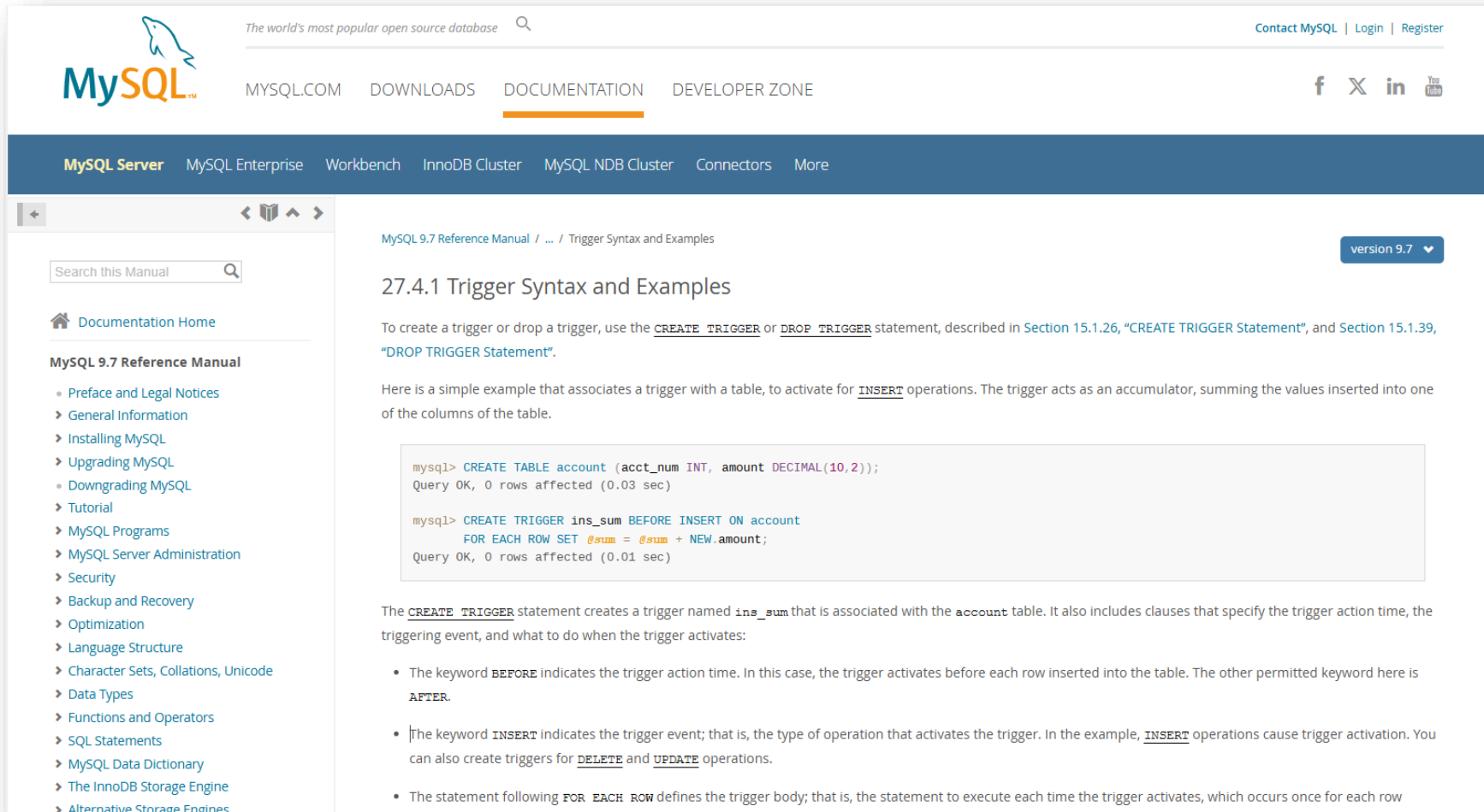
INSERT INTO animal_count (animals) VALUES(0);
```

We want to increment a counter each time a new animal is added. Here's what the trigger looks like:

```
CREATE TRIGGER increment_animal
AFTER INSERT ON animals
FOR EACH ROW
UPDATE animal_count SET animal_count.animals = animal_count.animals+1;
```


5. Triggers

Triggers exist in MariaDB and MySQL:



The screenshot shows the MySQL 9.7 Reference Manual page for "Trigger Syntax and Examples". The page is part of the "MySQL 9.7 Reference Manual" and is titled "27.4.1 Trigger Syntax and Examples". It includes a search bar, a navigation menu, and a list of contents. The main content area explains how to create a trigger using the `CREATE TRIGGER` statement, with a simple example provided in a code block. The example creates a table named `account` with columns `acct_num` (INT) and `amount` (DECIMAL(10,2)), and then creates a trigger named `ins_sum` that activates before each `INSERT` operation on the `account` table, summing the `amount` column.

MySQL 9.7 Reference Manual / ... / Trigger Syntax and Examples

version 9.7

27.4.1 Trigger Syntax and Examples

To create a trigger or drop a trigger, use the `CREATE TRIGGER` or `DROP TRIGGER` statement, described in Section 15.1.26, "CREATE TRIGGER Statement", and Section 15.1.39, "DROP TRIGGER Statement".

Here is a simple example that associates a trigger with a table, to activate for `INSERT` operations. The trigger acts as an accumulator, summing the values inserted into one of the columns of the table.

```
mysql> CREATE TABLE account (acct_num INT, amount DECIMAL(10,2));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TRIGGER ins_sum BEFORE INSERT ON account
FOR EACH ROW SET @sum = @sum + NEW.amount;
Query OK, 0 rows affected (0.01 sec)
```

The `CREATE TRIGGER` statement creates a trigger named `ins_sum` that is associated with the `account` table. It also includes clauses that specify the trigger action time, the triggering event, and what to do when the trigger activates:

- The keyword `BEFORE` indicates the trigger action time. In this case, the trigger activates before each row inserted into the table. The other permitted keyword here is `AFTER`.
- The keyword `INSERT` indicates the trigger event; that is, the type of operation that activates the trigger. In the example, `INSERT` operations cause trigger activation. You can also create triggers for `DELETE` and `UPDATE` operations.
- The statement following `FOR EACH ROW` defines the trigger body; that is, the statement to execute each time the trigger activates, which occurs once for each row

6. Views

In a relational database, a **view** is a **virtual table** based on the **result set of a SQL query**. Unlike regular tables, **views do not store data** themselves; instead, they dynamically retrieve data from one or more underlying tables whenever the view is queried.

Key Characteristics of Views

Virtual: Views do not store data; they are just saved SQL queries.

Dynamic: The data in a view is always up-to-date, reflecting changes in the underlying tables.

Simplification: Views can hide complex queries, joins, or columns, making data access easier for users.

Security: Views can restrict access to certain columns or rows, showing only what a user is allowed to see.

Common Uses of Views

Simplify complex queries (e.g., joining multiple tables).

Restrict data access (e.g., show only certain columns to specific users).

Present aggregated data (e.g., summaries, reports).

6. Views

SERVER USAGE > VIEWS

Edit

CREATE VIEW

Complete CREATE VIEW reference: OR REPLACE/IF NOT EXISTS, ALGORITHM=MERGE|TEMPTABLE|UNDEFINED, DEFINER/CURRENT_USER, and SQL SECURITY DEFINER|INVOKER.

Syntax

```
CREATE
[OR REPLACE]
[ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]
[DEFINER = { user | CURRENT_USER | role | CURRENT_ROLE }]
[SQL SECURITY { DEFINER | INVOKER }]
VIEW [IF NOT EXISTS] view_name [(column_list)]
AS select_statement
[WITH [CASCADED | LOCAL] CHECK OPTION]
```

Description

The `CREATE VIEW` statement creates a new [view](#), or replaces an existing one if the `OR REPLACE` clause is given. If the view does not exist, `CREATE OR REPLACE VIEW` is the same as `CREATE VIEW`. If the view does exist, `CREATE OR REPLACE VIEW` is the same as [ALTER VIEW](#).

The `select_statement` is a [SELECT](#) statement that provides the definition of the view. (When you select from the view, you select in effect using the `SELECT` statement.) `select_statement` can select from base tables or other views.

6. Views

```
-- Create the employees table
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    phone VARCHAR(20),
    hire_date DATE,
    salary DECIMAL(10, 2)
);

-- Insert 5 employees
INSERT INTO employees VALUES
(1, 'John', 'Doe', 'john.doe@example.com', '555-1234', '2020-05-15', 75000.00),
(2, 'Jane', 'Smith', 'jane.smith@example.com', '555-5678', '2019-11-20', 80000.00),
(3, 'Michael', 'Johnson', 'michael.j@example.com', '555-9012', '2021-03-10', 68000.00),
(4, 'Emily', 'Williams', 'emily.w@example.com', '555-3456', '2018-07-30', 90000.00),
(5, 'David', 'Brown', 'david.b@example.com', '555-7890', '2022-01-05', 72000.00);
```

```
-- Create a view for employee contact info
CREATE VIEW employee_contact_info AS
SELECT employee_id, first_name, last_name, email, phone
FROM employees;
```

```
-- Use the view to retrieve data
SELECT * FROM employee_contact_info;
```

6. Views

```
-- Create a view for employee contact info  
CREATE VIEW employee_contact_info AS  
SELECT employee_id, first_name, last_name, email, phone  
FROM employees;
```

```
CREATE USER 'hr_user'@'localhost' IDENTIFIED BY 'password';
```

```
GRANT SELECT ON employee_contact_info TO 'hr_user'@'localhost';
```

```
SHOW GRANTS FOR 'hr_user'@'localhost';
```

6. Views

```
PS C:\Users\lanjer> docker start my-mariadb4
my-mariadb4
PS C:\Users\lanjer> docker exec -it my-mariadb4 bash
root@156966479d18:/# |
root@156966479d18:/# mariadb -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 6
Server version: 12.2.2-MariaDB-ubu2404 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

6. Views

```
MariaDB [Company]> -- Create the employees table
MariaDB [Company]> CREATE TABLE employees (
    ->     employee_id INT PRIMARY KEY,
    ->     first_name VARCHAR(50),
    ->     last_name VARCHAR(50),
    ->     email VARCHAR(100),
    ->     phone VARCHAR(20),
    ->     hire_date DATE,
    ->     salary DECIMAL(10, 2)
    -> );
Query OK, 0 rows affected (0.023 sec)

MariaDB [Company]>
MariaDB [Company]> -- Insert 5 employees
MariaDB [Company]> INSERT INTO employees VALUES
    -> (1, 'John', 'Doe', 'john.doe@example.com', '555-1234', '2020-05-15', 75000.00),
    -> (2, 'Jane', 'Smith', 'jane.smith@example.com', '555-5678', '2019-11-20', 80000.00),
    -> (3, 'Michael', 'Johnson', 'michael.j@example.com', '555-9012', '2021-03-10', 68000.00),
    -> (4, 'Emily', 'Williams', 'emily.w@example.com', '555-3456', '2018-07-30', 90000.00),
    -> (5, 'David', 'Brown', 'david.b@example.com', '555-7890', '2022-01-05', 72000.00);
Query OK, 5 rows affected (0.008 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

6. Views

```
MariaDB [Company]> -- Create a view for employee contact info
MariaDB [Company]> CREATE VIEW employee_contact_info AS
    -> SELECT employee_id, first_name, last_name, email, phone
    -> FROM employees;
Query OK, 0 rows affected (0.007 sec)
```

```
MariaDB [Company]> -- Use the view to retrieve data
MariaDB [Company]> SELECT * FROM employee_contact_info;
```

employee_id	first_name	last_name	email	phone
1	John	Doe	john.doe@example.com	555-1234
2	Jane	Smith	jane.smith@example.com	555-5678
3	Michael	Johnson	michael.j@example.com	555-9012
4	Emily	Williams	emily.w@example.com	555-3456
5	David	Brown	david.b@example.com	555-7890

5 rows in set (0.002 sec)

6. Views

```
MariaDB [Company]> CREATE USER 'hr_user'@'localhost' IDENTIFIED BY 'pw4321';  
Query OK, 0 rows affected (0.004 sec)
```

```
MariaDB [Company]> GRANT SELECT ON employee_contact_info TO 'hr_user'@'localhost';  
Query OK, 0 rows affected (0.003 sec)
```

```
MariaDB [Company]> SHOW GRANTS FOR 'hr_user'@'localhost';
```

```
+-----+  
| Grants for hr_user@localhost |  
+-----+  
| GRANT USAGE ON *.* TO `hr_user`@`localhost` IDENTIFIED BY PASSWORD '*A5F097C110FFF44A0B48862555E6284B5C36FDAB' |  
| GRANT SELECT ON `Company`.`employee_contact_info` TO `hr_user`@`localhost` |  
+-----+  
2 rows in set (0.000 sec)
```

6. Views

```
MariaDB [Company]> quit
Bye
root@156966479d18:/# mariadb -u hr_user -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 7
Server version: 12.2.2-MariaDB-ubu2404 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> use Company;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [Company]> SELECT * FROM employees;
ERROR 1142 (42000): SELECT command denied to user 'hr_user'@'localhost' for table 'Company`.`employees`
MariaDB [Company]> SELECT * FROM employee_contact_info;
```

employee_id	first_name	last_name	email	phone
1	John	Doe	john.doe@example.com	555-1234
2	Jane	Smith	jane.smith@example.com	555-5678
3	Michael	Johnson	michael.j@example.com	555-9012
4	Emily	Williams	emily.w@example.com	555-3456
5	David	Brown	david.b@example.com	555-7890

```
5 rows in set (0.001 sec)
```

6. Views

```
MariaDB [Company]> quit  
Bye  
root@156966479d18:/# exit  
exit
```

ANY QUESTIONS?

BE POSITIVE, B+!



JÖNKÖPING UNIVERSITY